

BÁO CÁO TÓM TẮT NGHIÊN CỨU

Benchmark data-quality alerting / observability bằng fault injection

Đề tài: Xây dựng benchmark có tính tái tạo cho việc đánh giá hệ thống cảnh báo chất lượng dữ liệu trên dữ liệu dạng bảng (tabular data), sử dụng fault injection có kiểm soát và đánh giá theo sự cố (incident-aware evaluation).

1. Mục đích dự án

Ta **tạo protocol** (thước đo) rồi dùng nó để **đo các phương pháp phát hiện** (baselines).

Mục đích cuối cùng - trả lời các câu hỏi chưa ai trả lời:

- Loại lỗi Data Quality nào dễ phát hiện, loại nào khó?
- Phương pháp nào mạnh ở đâu, yếu ở đâu?
- Nếu tính cả tốc độ phát hiện thì ranking có thay đổi không?
- Các tool hiện tại (Great Expectations) bỏ sót gì?

Đóng góp = cả bài thi (benchmark protocol) lẫn kết quả phân tích (regime analysis).

2. Papers đã nghiên cứu, họ đã làm gì, và gap ở đâu?

Nhóm 1: Hệ thống kiểm tra chất lượng dữ liệu (Data quality validation)

Schelter et al. (2018) - Deequ:

Xây dựng hệ thống kiểm tra Data quality trên Apache Spark. Người dùng định nghĩa các ràng buộc dạng unit-test cho DataFrame (ví dụ: cột X không được null quá 5%, cột Y phải unique). Deequ tự động tính toán metric Data quality, hỗ trợ gợi ý ràng buộc từ data profiling, và có thể chạy tăng dần bằng sketches.

=> **Họ giải quyết:** chạy check Data quality ở production scale.

=> **Họ KHÔNG giải quyết:** đánh giá chất lượng cảnh báo. Deequ nói "check này fail", nhưng không nói "hệ thống cảnh báo phát hiện sự cố nhanh hay chậm, chính xác hay không."

=> **Ta kế thừa:** Great Expectations (tool thuộc cùng họ phương pháp constraint/expectation-based với Deequ, nhưng không phải implementation từ paper Deequ) được ta dùng làm external baseline - chạy tool thật qua benchmark để đo.

Redyuk et al. (2021) - Recurring/Dynamic Ingestion Validation:

Mở rộng Data quality validation cho luồng dữ liệu liên tục, sử dụng lịch sử batch trước để phát hiện bất thường.

=> **Họ giải quyết:** Data quality check có tính thời gian, dựa trên lịch sử.

=> **Họ KHÔNG giải quyết:** benchmark chuẩn hóa để so sánh các phương pháp phát hiện.

=> **Ta kế thừa:** ý tưởng dùng lịch sử batch để detect anomaly - đây là cơ sở cho history-based statistical baseline của ta.

Nhóm 2: Benchmark đánh giá phát hiện bất thường (anomaly detection evaluation)

Lavin & Ahmad (2015) - NAB (Numenta Anomaly Benchmark):

Benchmark đầu tiên cho anomaly detection trên time series có tính đến thời gian. Đưa ra khái niệm detection window, sigmoid reward (phát hiện sớm thường nhiều hơn phát hiện muộn), và application profiles (3 bộ trọng số khác nhau cho FP/FN).

=> **Họ giải quyết:** đánh giá delay-aware, không chỉ đúng/sai mà còn nhanh/chậm.

=> **Họ KHÔNG giải quyết:** chỉ cho time series 1 chiều. Không có bảng, cột, quan hệ khóa ngoại.

=> Ta kế thừa: Ta kế thừa từ NAB ý tưởng anomaly windows, early-detection credit cho phát hiện sớm nhất trong mỗi window, và application profiles cho các trade-off FP/FN.

Tatbul et al. (2018) - Range-based Precision/Recall:

Đề xuất precision/recall cho anomaly dạng đoạn liên tục. Tính overlap giữa predicted range và ground truth range, có position bias và cardinality penalty.

=> **Họ giải quyết:** metric tốt hơn cho anomaly kéo dài.

=> **Ta kế thừa:** ý tưởng overlap/range semantics từ Tatbul để xây matching protocol cho tabular DQ; thay vì match theo đoạn thời gian, ta match theo bảng/cột/quan hệ.

Paparrizos et al. (2022) - TSB-UAD:

Benchmark engineering ở quy mô lớn cho univariate time-series anomaly detection (TSAD).

=> **Họ giải quyết:** xây benchmark quy mô lớn với nhiều detector trong một harness thống nhất, giúp so sánh kết quả nhất quán ở cấp hệ benchmark.

=> **Ta kế thừa:** bài học benchmark engineering và calibration fairness - ta thiết kế calibration protocol (percentile-based + alert-budget) để tránh threshold bias, và dùng pattern unified evaluation harness tương tự cho 4 baselines.

Paparrizos et al. (2022) - VUS:

Metric family threshold-independent / parameter-free, cho thấy lựa chọn metric có thể làm ranking thay đổi.

=> **Họ giải quyết:** đưa ra họ metric giúp giảm phụ thuộc vào việc chọn threshold đơn lẻ khi đánh giá detector.

=> **Họ KHÔNG giải quyết:** vẫn ở bối cảnh univariate time series, không xử lý alerting trên tabular DQ.

=> **Ta kế thừa:** bài học rằng lựa chọn metric có thể làm đổi ranking, nên benchmark của ta phải có protocol calibration và reporting đủ chặt thay vì dựa vào một ngưỡng tùy ý.

Nhóm 3: Benchmark injection lỗi và data cleaning

Schelter et al. (2021) - Jenga:

Framework inject lỗi có kiểm soát vào dữ liệu bảng: missing values, noise, scaling errors. Đo tác động lên ML model.

=> **Họ giải quyết:** fault injection + đo downstream ML impact.

=> **Họ KHÔNG giải quyết:** không đánh giá hệ thống cảnh báo. Jenga hỏi "lỗi làm model tệ bao nhiêu?", không hỏi "tool cảnh báo phát hiện lỗi nhanh chậm ra sao?"

=> **Ta kế thừa:** Ta kế thừa từ Jenga kỹ thuật controlled corruption/fault injection và triết lý repeated stress-testing; không kế thừa detector baseline từ Jenga.

Abdelaal et al. (2023) - REIN:

Benchmark framework cho error detection và repair methods trong ML pipelines.

=> Họ giải quyết: benchmark hóa việc so sánh các phương pháp error detection/repair trong ML pipelines dưới một framework thống nhất.

=> **Họ KHÔNG giải quyết:** đánh giá ở cấp incident/alert. Không có delay, duplicate burden, FP analysis.

=> Ta kế thừa: Ta kế thừa mô hình benchmark-controller / unified evaluation harness, không kế thừa trực tiếp alert metrics.

Gap tổng hợp

Mảnh ghép có sẵn	Thiếu gì cho DQ alerting trên bảng?
NAB, Tatbul, VUS — đánh giá delay-aware	Chỉ cho time series, không có entity-aware matching (bảng/cột/quan hệ)
Deequ, Great Expectations — chạy check DQ	Chỉ check pass/fail, không đánh giá chất lượng cảnh báo
Jenga, REIN — inject lỗi có kiểm soát	Đánh giá ML robustness hoặc cell-level cleaning, không phải alerting

Gap statement: Chưa có benchmark nào kết hợp:

(1) fault injection có kiểm soát

(2) ground truth cấp sự cố

(3) matching alert-to-incident formal

(4) đánh giá delay-aware + localization-aware

(5) đo false positive trên dữ liệu sạch - cho Data quality alerting trên dữ liệu bảng.

3. Đóng góp

Điều mới chưa ai làm

1. **Matching protocol formal cho tabular DQ** — thuật toán GreedyEarliestMatch, kế thừa detection window từ NAB và lấy cảm hứng từ overlap/range semantics của Tatbul để xây entity-aware matching trên bảng/cột/quan hệ.
 2. **Bộ 8 metric đánh giá đa chiều — không chỉ đúng/sai mà đo cả:** delay (từ NAB), localization (mới), duplicate burden (thiết kế của đề tài, lấy cảm hứng từ alert fatigue trong vận hành), FP rate trên dữ liệu sạch (thiết kế của đề tài để kiểm tra false positive). Mỗi metric có công thức toán học. Không dùng composite score
 3. **Regime analysis map** — cho thấy loại lỗi DQ nào dễ/khó phát hiện. Kiến thức thực tiễn mới, chưa có paper nào công bố.
-

4. Định hướng chi tiết — Scope, baselines, stack, flow

Mô tả

Xây dựng benchmark: inject lỗi có kiểm soát vào dữ liệu bảng sạch → ghi ground truth → chạy 4 detector baselines → đo bằng 8 metric → tổng hợp regime analysis map.

Scope

- 5 fault families x 3 severity x 2 duration x 5 seeds → ~**150 dirty runs** + 15-20 clean runs
- 4 baselines, 5 ablation bắt buộc + 3 mở rộng
- Artifact công khai: toàn bộ code, config, seed, script

4 baselines — căn cứ khoa học và code thực tế

Nguyên tắc: **dùng code thật từ paper/tool gốc**, không viết version đơn giản hóa. Mỗi baseline đại diện cho một **lớp phương pháp** khác nhau.

Baseline	Đại diện cho lớp nào	Căn cứ khoa học chính	Code thực tế
Constraint-based validation	Kiểm tra DQ bằng rule/constraint/expectation	Deequ (Schelter et al., 2018) — hệ thống constraint-based validation trên DataFrame: <i>isComplete, isUnique, isInRange, hasCompleteness...</i> paper	Dùng Great Expectations (pip install) — thuộc cùng họ phương pháp constraint/expectation-based với Deequ, nhưng không phải implementation từ paper Deequ. Không viết lại logic — dùng tool thật + adapter
History-based batch monitoring	Phát hiện lệch khỏi lịch sử bằng đặc trưng thống kê theo batch	Redyuk et al. (2021) — <i>tính quality metrics per batch (completeness, distinct count, min/max/mean/std, ratio of most frequent) → dùng kNN novelty detection</i> paper	<i>Implement theo đúng approach của paper: tính batch quality profile → novelty detection (kNN-based như paper mô tả).</i>
Unsupervised outlier detection	ML không giám sát cho phát hiện anomaly trên vector metric	Isolation Forest (Liu et al., 2008) — thuật toán gốc: xây isolation trees, anomaly score = inverse path length paper. Được dùng như thuật toán gốc cho unsupervised anomaly detection; implementation dùng scikit-learn (IsolationForest)	Dùng scikit-learn (IsolationForest) — implementation chuẩn của thuật toán gốc. Không viết lại
Naive threshold baseline	Tham chiếu đơn giản nhất — nếu cái này đã đủ tốt thì không cần ML	Lower bound của benchmark: static threshold trên từng metric (null_ratio > X, dup_rate > Y)	

Lưu ý quan trọng:

- Baseline **Great Expectations** và **Redyuk** approach là hai cách khác nhau:
 - + **Great Expectations** check static constraints mỗi batch

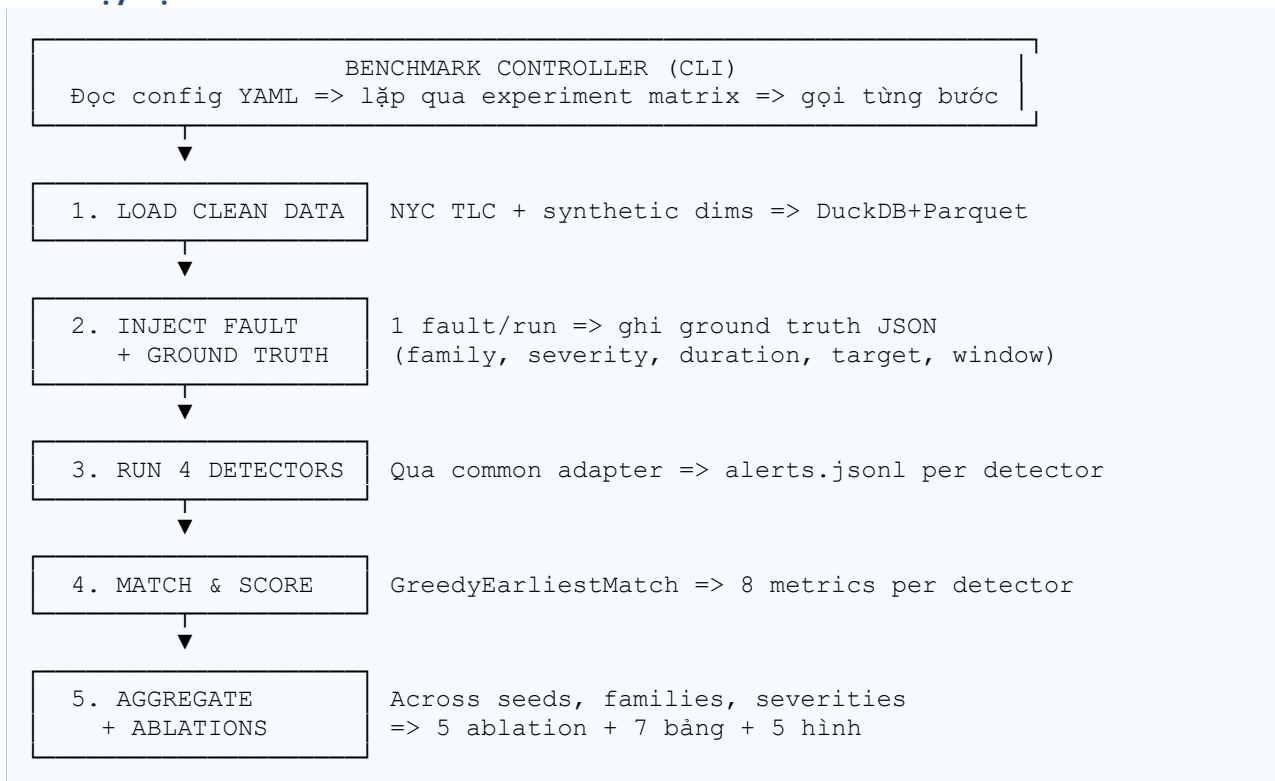
+ **Redyuk** so sánh batch hiện tại với lịch sử bằng novelty detection

- **Isolation Forest** fit trên clean calibration windows → predict trên dirty runs → output anomaly score → convert thành alert → chấm bằng 8 metric. Không cần labels, không cần GPU
- **Naive threshold** là lower bound — nếu baselines 1-3 không hơn được cái này thì chúng không có giá trị
- NAB (Lavin & Ahmad, 2015) là **evaluation framework**. Ta kế thừa cho scoring protocol

Stack kỹ thuật (First stage)

Layer	Component	Technology
Data	Storage + query	DuckDB + Parquet
Data	Dataset	NYC TLC public API + Python + Faker (synthetic dims)
Injection	Fault injector + ground truth	Python module, YAML config-driven
Detection	4 baselines	Great Expectations + Redyuk approach (kNN) + scikit-learn IF + naive threshold
Evaluation	Matcher + Scorer + Calibration	Python (GreedyEarliestMatch + 8 metrics)
Orchestration	Controller + Config	CLI Python + YAML + seed pinning
Reporting	Aggregator + Ablation runner	Pandas + matplotlib/seaborn

Flow chạy dự kiến



Tổng: ~170 runs x 4 baselines = ~680 lượt chấm điểm

5. Data, Injection, và vấn đề Overfit

Data từ đâu?

- **Dataset chính:** NYC TLC Yellow Taxi Trip Records (public, có time structure) — 2-3 tháng
- **Mở rộng synthetic:** 2-3 bảng dimension nhỏ (zone, payment type) — chỉ dùng cho FK-break
- **Không dùng TPC-DS:** quá nặng, cân nhắc

Inject cái gì?

Fault family	Mô tả	Duration
Null spike	Tăng đột ngột giá trị null	one-window + sustained
Duplicate burst	Trùng lặp hàng hoặc business key	
Range violation	Giá trị ngoài khoảng hợp lệ	
Freshness lag	Batch dữ liệu bị thiếu hoặc trễ	
Foreign-key break	Fact rows tham chiếu key không tồn tại	

Mỗi family: 3 severity (low/medium/high) x 2 duration.

Tự inject có bị overfit không?

Rủi ro: tự thiết kế lỗi + tự thiết kế detector = self-serving.

Cách giảm thiểu:

1. Data gốc public (NYC TLC) => phân phối không do tác giả thiết kế
2. External baseline (Great Expectations) => phá vỡ closed-loop
3. Synthetic chỉ cho FK-break - scope rất hẹp
4. 5 ablation kiểm chứng - chứng minh kết quả không phụ thuộc lựa chọn tùy ý
5. Hạn chế thừa nhận: điểm inject do ta chọn - cũng giống NAB, Jenga, REIN. Mitigation: công bố toàn bộ config, seed, script.

6. Metric tiềm năng?

Metric	Ý nghĩa	Kế thừa từ đâu
Incident Recall	Tỷ lệ sự cố được phát hiện	Chuẩn IR/AD literature
Incident Precision	Tỷ lệ alert đúng	Chuẩn, adapt cho incident-level
Incident F1	Trung bình hài hòa R/P	Chuẩn
Detection Delay	Bao nhiêu batch mới phát hiện	NAB (Lavin & Ahmad, 2015) — detection window concept
Localization Accuracy	Alert đúng cột/bảng/quan hệ không	Mới — đặc thù cho tabular DQ

Metric	Ý nghĩa	Kế thừa từ đâu
		(không có trong TSAD)
Duplicate Burden	Bao nhiêu alert thừa	Thiết kế của đề tài, lấy cảm hứng từ alert fatigue trong vận hành
Clean-Run FP Rate	False positive trên data sạch	Thiết kế của đề tài để kiểm tra false positive trên dữ liệu sạch
Runtime Overhead	Thời gian chạy detector	Chuẩn benchmark

7. Kết quả dự kiến

RQ	Dự kiến
RQ1: Loại lỗi nào dễ/khó?	Null spike, range violation dễ; freshness lag, FK break khó hơn cho rule-based
RQ2: Delay-aware thay đổi ranking?	Ít nhất 1 cặp baseline đổi thứ hạng khi tính delay + FP
RQ3: FP trên dữ liệu sạch?	FP rate khác nhau đáng kể giữa các baseline
RQ4: Kết luận ổn định?	Ranking không đổi qua 5 seeds và 3 calibration policies

Output: 7 bảng + 5 hình + regime analysis map + public artifact (GitHub repo)

8. Nơi có thể gửi paper

Workshop

Venue		Deadline	Phù hợp
QDB 2026 @ VLDB , Boston	8-9/2026	~5/2026	Chuyên data quality — đúng domain nhất
DBTest 2026 @ SIGMOD	5-6/2026	Check website	Testing + benchmarking. ACM DL
DEEM 2027 @ SIGMOD	~6/2027	~2-3/2027	Data management cho ML pipelines

Conference

Venue	Khi nào	Deadline	Phù hợp
KDD 2027 Benchmarks Track	8/2027	~7/2026	Chuyên benchmark papers

Venue	Khi nào	Deadline	Phù hợp
VLDB 2027	9/2027	Rolling	Hàng đầu data management
SIGMOD 2027	2027	Round 1: 1/2026	Top venue

Lộ trình

Q2 2026: Hoàn thành experiments

- 5/2026: QDB 2026 @ VLDB (workshop, đúng domain)
- 7/2026: KDD 2027 Benchmarks Track (prestige)

▼

Q3-Q4 2026: Nhận review => strengthen

▼

Q1 2027: Version mạnh nhất => SIGMOD 2027 / VLDB 2027